

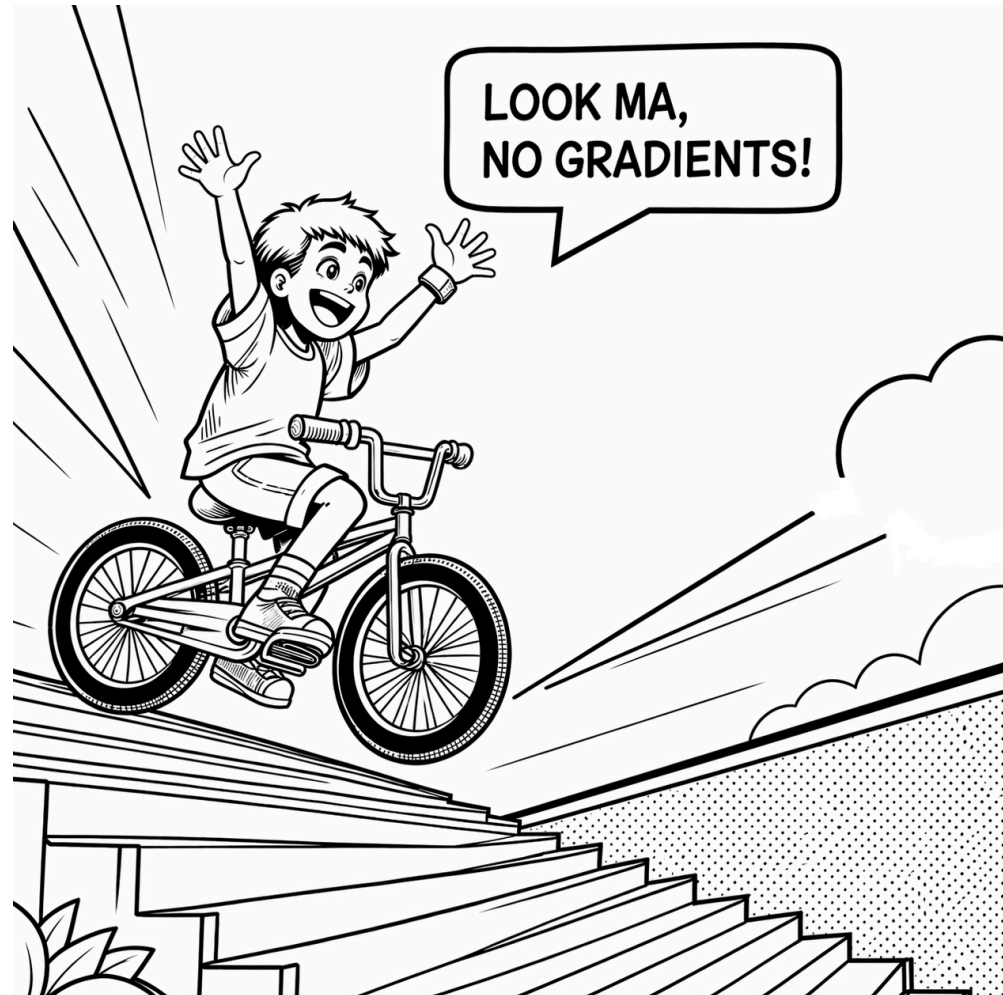
Optimierungsverfahren

Teil 05 - Gradientenfrei

Martin Zaefferer

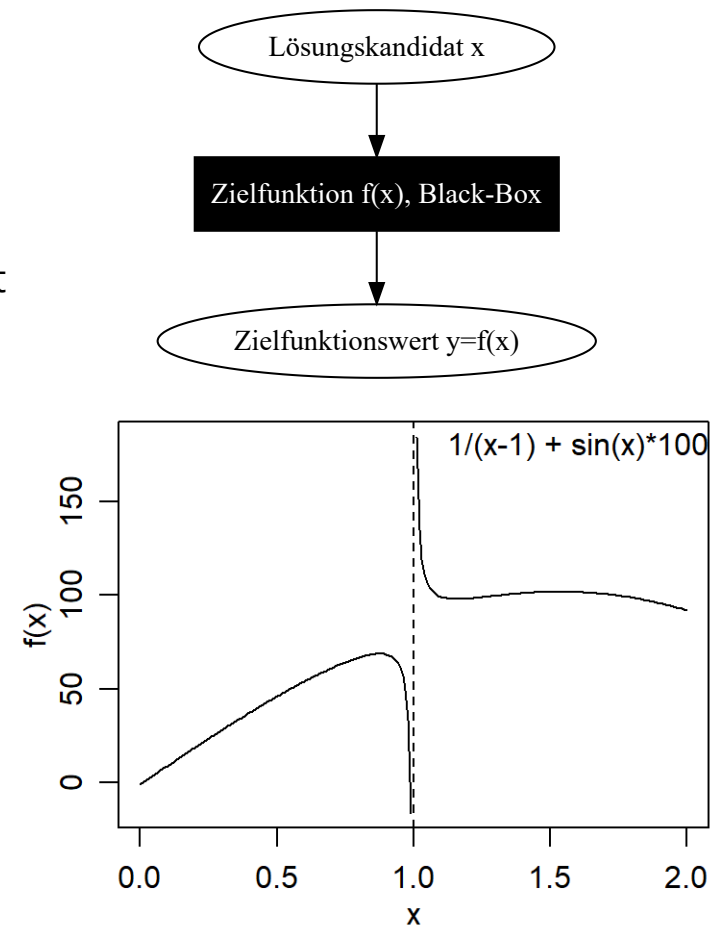
Bild mit KI-Unterstützung erzeugt.

Keine Gradienten!



Gradientenfreie Optimierung

- Bisher: Algorithmen brauchen Gradienten, Hessematrix
- Viele Probleme: nicht differenzierbar
 - Black-box Probleme
 - Innenleben / Details der Zielfunktion unbekannt
 - Wissensgewinn nur über: Auswertung von $f(\vec{x})$
 - Diskontinuierlich Probleme
- Lösungsansatz: Gradientenfreie Optimierung



Überblick bis hier

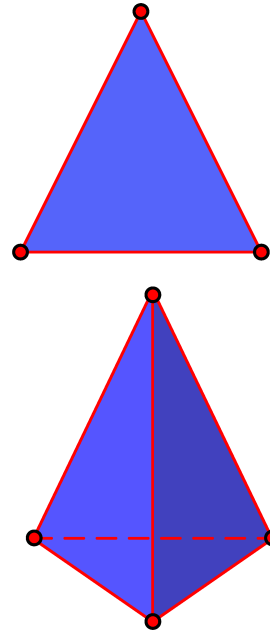
Eigenschaften von Algorithmen/Methoden

Methode	Global	Gradientenbasiert	Nebenbedingungen	Ziele	Linear	Ansatz	Genauigkeit	Deterministisch	Variablen
Stationäre Punkte / Hesse	✓	✓	ohne	1	✗	analytisch	exakt	✓	$\mathbb{R}^n$
Gradient Descent	✗	✓	ohne	1	✗	numerisch	?	✓	$\mathbb{R}^n$
Simplex	✗	✓	alle	1	✓	numerisch	exakt	✓	$\mathbb{R}^n$ & $\mathbb{Z}^n$
QO	✗	✓	alle	1	Nebenbed.	numerisch	?	✓	$\mathbb{R}^n$
KKT	✓	✓	alle	1	✗	analytisch	exakt	✓	$\mathbb{R}^n$
SQO	✗	✓	alle	1	✗	numerisch	?	✓	$\mathbb{R}^n$

Algorithmus: Nelder-Mead Simplex

Nelder-Mead Simplex

- Simplex:
 - 1-D Simplex: Liniensegment
 - 2-D Simplex: Dreieck
 - 3-D Simplex: Tetrahedron
 - n -D Simplex: Polytop mit $n + 1$ Eckpunkten



Nelder-Mead Simplex: Algorithmus

Gegeben: initiale Lösung $\vec{x}_0$ im n-dimensionalen Raum

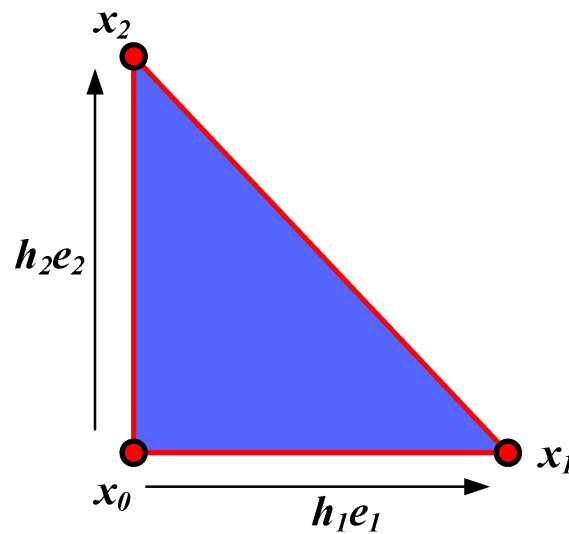
1. Erzeuge initialen Simplex, $\vec{x}_0, \vec{x}_1, \vec{x}_2, \dots, \vec{x}_n$
2. Sortiere die Ecken nach Zielfunktionswert $f(\vec{x})$
3. Wenn Abbruchbedingung erfüllt: Stopp. Sonst: weiter.
4. Berechne 'centerpoint' der besten Seite $\vec{c}$
5. Transformiere Simplex: a) reflect, b) expand, c) contract, d) shrink

J. A. Nelder, R. Mead. A Simplex Method for Function Minimization, The Computer Journal, Oxford University Press, 1965 , 7 , 308-313

Nelder-Mead: Erste Schritte

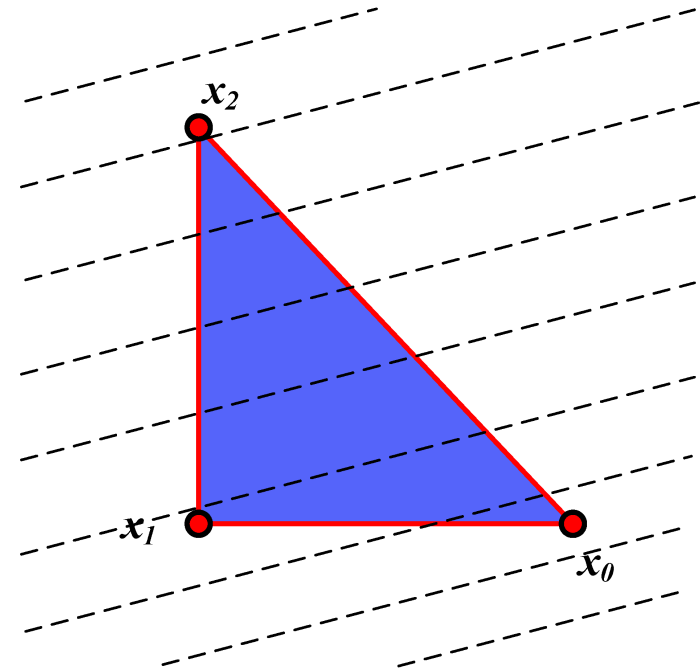
Nelder-Mead Simplex: 1 Initialisierung

- Gegeben: Initiallösung $\vec{x}_0$
- $\vec{x}_j = \vec{x}_0 + h_j \vec{e}_j$
- mit
 - $j = 1, \dots, n$
 - Schrittweite h_j
 - j -th Einheitsvektor $\vec{e}_j$



Nelder-Mead Simplex: 2 Sortierung

- Sortiere die Indizes: $f(\vec{x}_0) \leq f(\vec{x}_1) \leq \dots \leq f(\vec{x}_n)$



Nelder-Mead Simplex: 3 Abbruchbedingung

- Abbruch wenn
 - Summe der Distanzen zwischen Eckpunkten

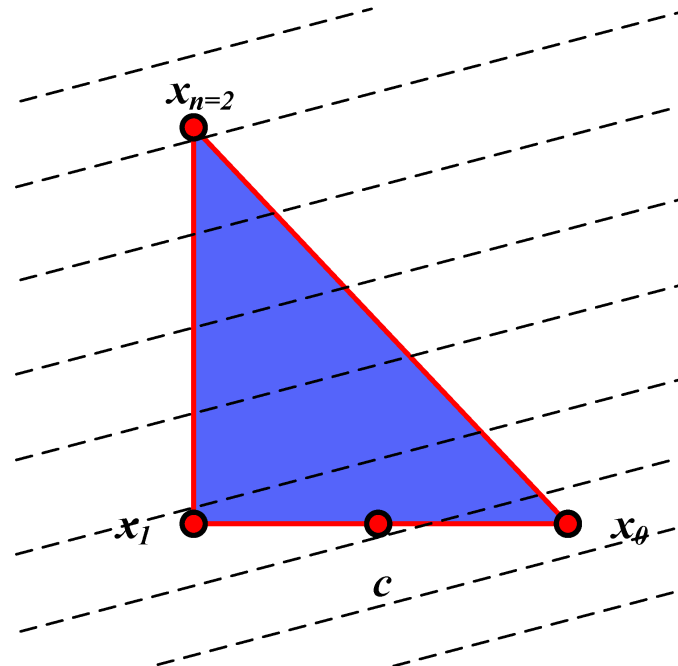
$$\sum_i^n \sum_{j \neq i}^n ||\vec{x}_j - \vec{x}_i||$$

wird klein (unter Schwellenwert)

- Differenz $f(\vec{x}_n) - f(\vec{x}_0)$ wird klein (unter Schwellenwert)
- Erreicht maximale Zahl von Auswertungen / Iterationen

Nelder-Mead Simplex: 4 centerpoint

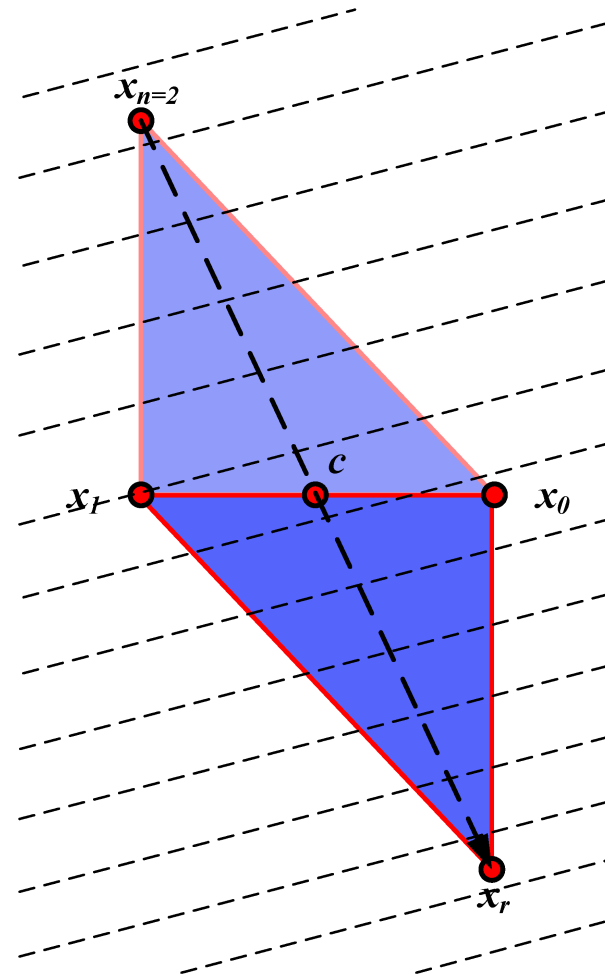
- Berechne centerpoint der 'besten Seite'
- d.h., die Seite gegenüber von $\vec{x}_n$
- $\vec{c} = \frac{1}{n} \sum_{j=0}^{n-1} \vec{x}_j$



Nelder-Mead: Transformationsschritte

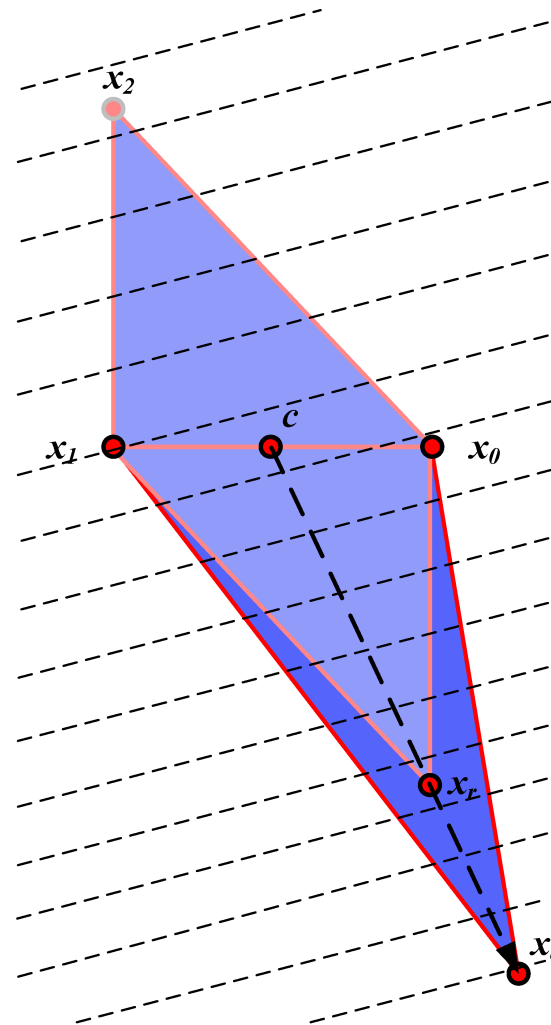
Nelder-Mead Simplex: 5a) reflect

- Erzeuge neuen Punkt mit Reflektion
- $\vec{x}_r = \vec{c} + \alpha(\vec{c} - \vec{x}_n)$
- Wenn $f(\vec{x}_0) \leq f(\vec{x}_r) \leq f(\vec{x}_{n-1})$
 - Ersetze Schlechtesten $\vec{x}_n = \vec{x}_r$
 - Gehe zu Schritt 2
- Wenn $f(\vec{x}_r) < f(\vec{x}_0)$
 - Weiter mit 5b) expand
- Wenn $f(\vec{x}_r) \geq f(\vec{x}_{n-1})$
 - Weiter mit 5c) contract



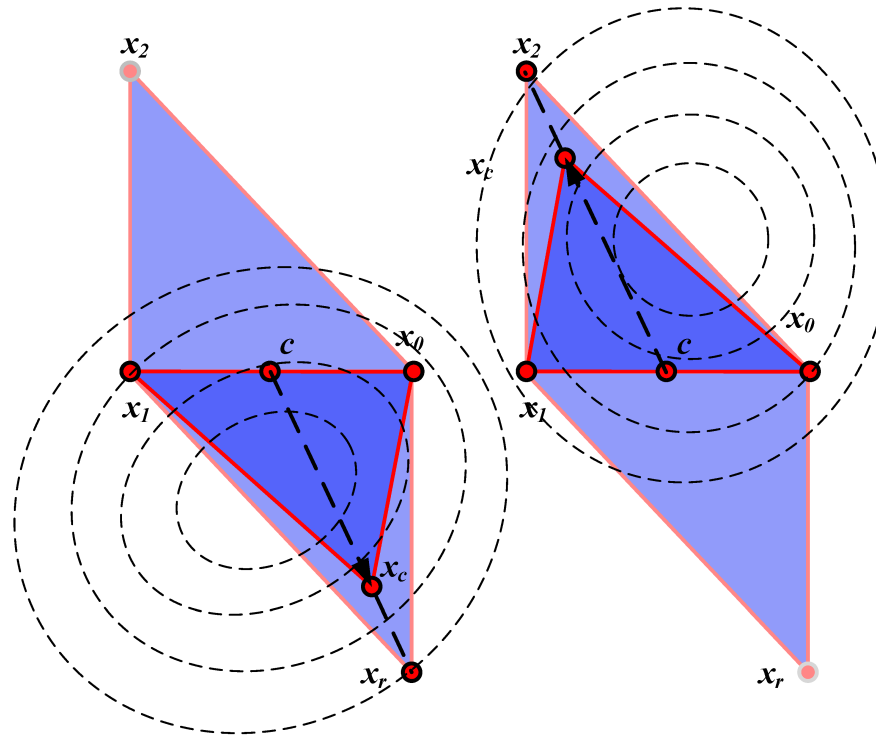
Nelder-Mead Simplex: 5b) expand

- Erzeuge neuen Punkt mit Erweiterung (expand)
- $\vec{x}_e = \vec{c} + \gamma(\vec{x}_r - \vec{c})$
- Wenn $f(\vec{x}_e) < f(\vec{x}_r)$
 - Ersetze Schlechtesten $\vec{x}_n = \vec{x}_e$
 - Gehe zu Schritt 2
- Sonst
 - Ersetze Schlechtesten $\vec{x}_n = \vec{x}_r$
 - Gehe zu Schritt 2



Nelder-Mead Simplex: 5c) contract

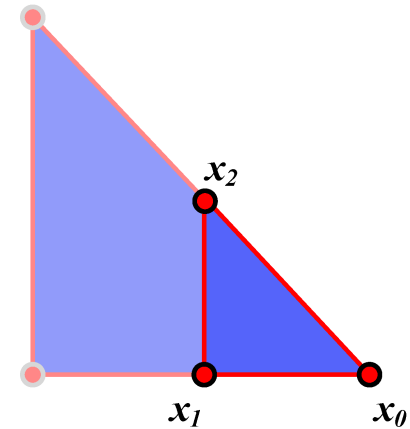
- Wenn $f(\vec{x}_{n-1}) \leq f(\vec{x}_r) < f(\vec{x}_n)$
 - Kontrahiere mit $\vec{x}_r$
 - $\vec{x}_c = \vec{c} + \beta(\vec{x}_r - \vec{c})$
- Sonst, wenn $f(\vec{x}_r) \geq f(\vec{x}_n)$
 - Kontrahiere mit $\vec{x}_n$
 - $\vec{x}_c = \vec{c} + \beta(\vec{x}_n - \vec{c})$
- Wenn $f(\vec{x}_c) < f(\vec{x}_n)$
 - Ersetze Schlechtesten $\vec{x}_n = \vec{x}_c$
 - Gehe zu Schritt 2
- Sonst weiter mit 5d) shrink



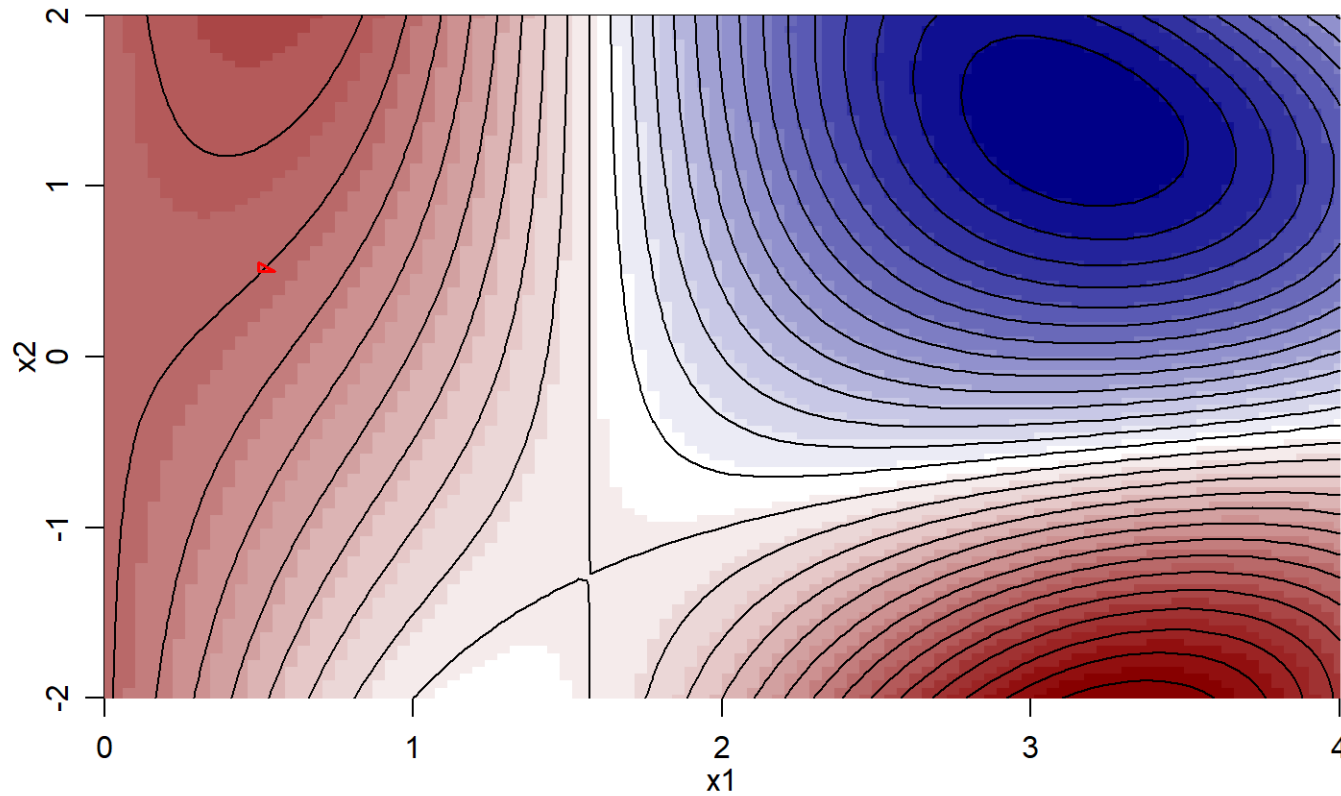
Nelder-Mead Simplex: 5d) shrink

Kontrahierter Punkt schlechter als $\vec{x}_n$: kann nur passieren, wenn Suchlandschaft komplexe Form annimmt (bzw. Simplex ist weit weg von lokalem Optimum). Deshalb: schrumpfen in Richtung des besten Punktes (Suchlandschaft wird 'einfacher').

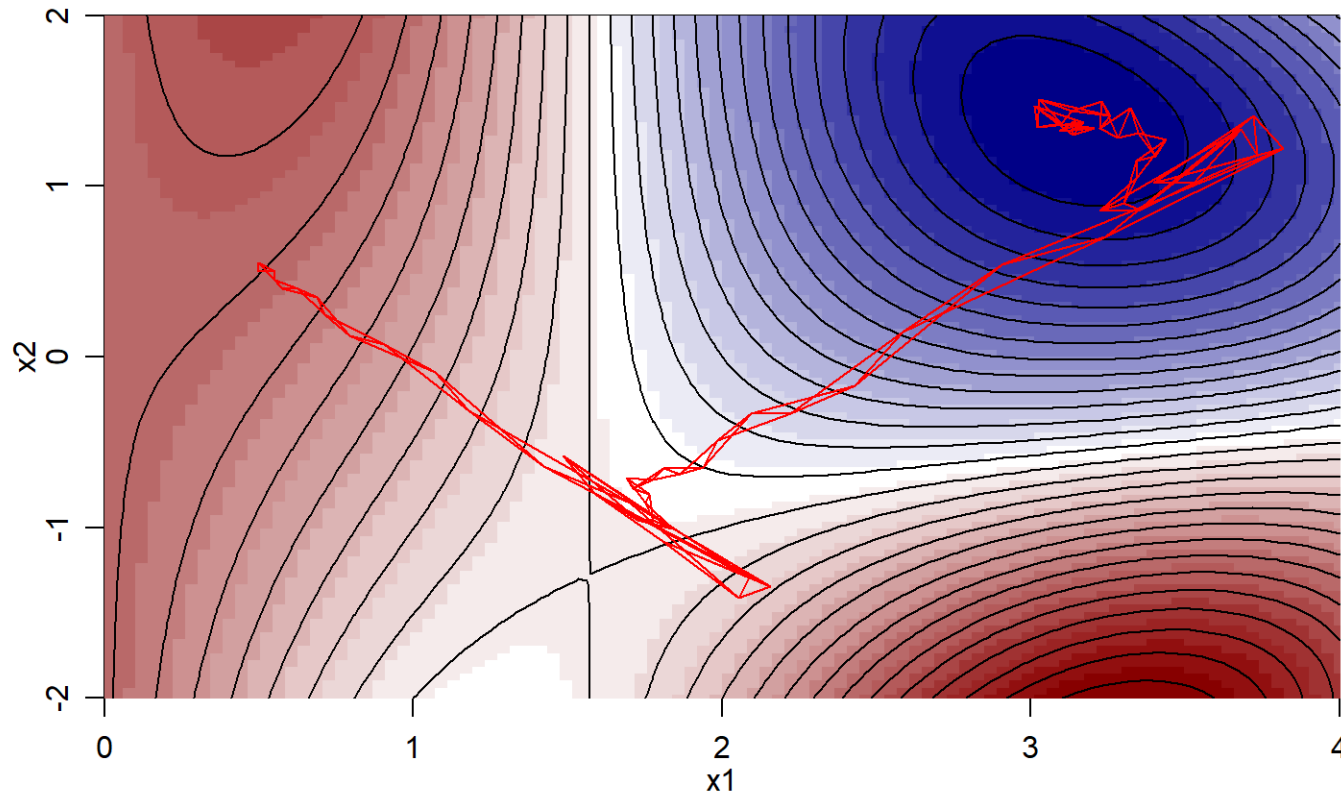
- $\vec{x}_j = \vec{x}_0 + \delta(\vec{x}_j - \vec{x}_0)$ für alle $j > 0$
- Gehe zu Schritt 2



Nelder-Mead Simplex - Demo



Nelder-Mead Simplex - Suchpfad



Überblick

Eigenschaften von Algorithmen/Methoden

Methode	Global	Gradientenbasiert	Nebenbedingungen	Ziele	Linear	Ansatz	Genauigkeit	Deterministisch	Variablen
Stationäre Punkte / Hesse	✓	✓	ohne	1	✗	analytisch	exakt	✓	$\mathbb{R}^n$
Gradient Descent	✗	✓	ohne	1	✗	numerisch	?	✓	$\mathbb{R}^n$
Simplex	✗	✓	alle	1	✓	numerisch	exakt	✓	$\mathbb{R}^n$ & $\mathbb{Z}^n$
QO	✗	✓	alle	1	Nebenbed.	numerisch	?	✓	$\mathbb{R}^n$
KKT	✓	✓	alle	1		analytisch	exakt	✓	$\mathbb{R}^n$
SQO	✗	✓	alle	1		numerisch	?	✓	$\mathbb{R}^n$
Nelder-Mead	✗	✗	ohne	1	✗	numerisch	approx.	✓	$\mathbb{R}^n$

R Anwendung

- Siehe Details in AnwendungOV-05a-NMS.Rmd
- Problem: Nelder-Mead ist lokal, nicht geeignet für multi-modale Probleme
- Aufgabe: Nelder-Mead als globale Suche implementieren, mit wiederholten Neustarts des Algorithmus
- Dann ausführen auf dem Beispielproblem, plotten aller gefundenen Optima

Algorithmus: DIRECT

Dividing RECTangles (DIRECT): Algorithmus

Wichtige Eigenschaften: **global**, nichtlinear, **Grenz-nebenbedingungen**, deterministisch, gradientenfrei, kontinuierlich, einziel

Gegeben: Suchgrenzen $\vec{x}_{lower}$ und $\vec{x}_{upper}$. Maximale Anzahl von Iterationen T . Dimensionalität n , und i -te Einheitsvektoren $\vec{e}_i$.

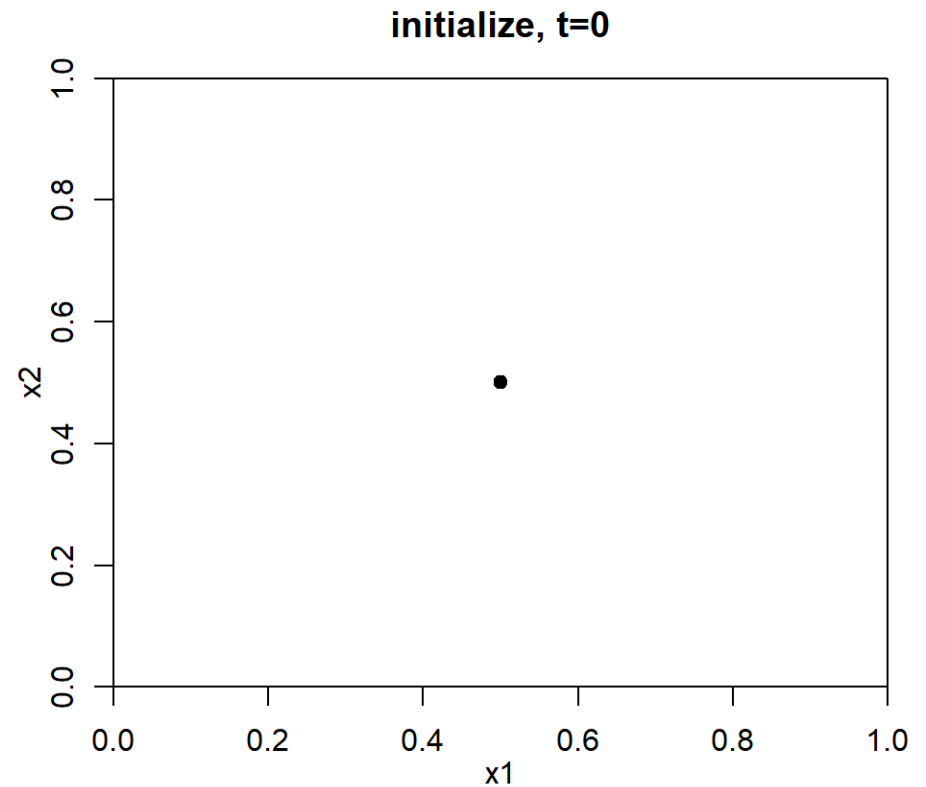
Algorithmus:

1. Initialisierung
2. Bestimmung der Menge S von potentiell optimalen Rechtecken
3. Für jedes Rechteck $j \in S$: sample & divide
4. $t = t + 1$, wenn $t = T$ stopp, sonst gehe zu Schritt 2.

Jones, D. R.; Perttunen, C. D. & Stuckman, B. E. Lipschitzian optimization without the Lipschitz constant, Journal of Optimization Theory und Applications, 1993 , 79 , 157-181

DIRECT Step 1: Initialisierung

- Normalisieren des Suchraums zu unit-hypercube ($x_i \in [0, 1]$, mit $i = 1, \dots, n$)
- centerpoint: $\vec{c}_0$
- Mit $f_{\min} = f(\vec{c}_0)$
- $m = 1$ (Auswertungszähler)
- $t = 0$ (Iterationszähler).



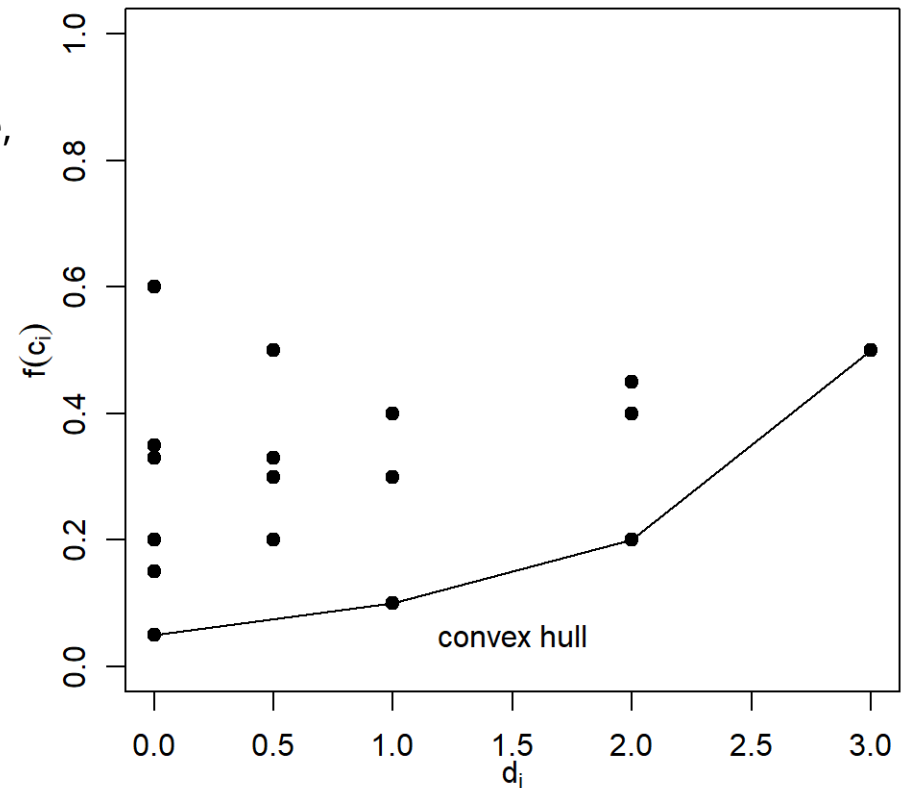
**DIRECT: potentiell optimale
Rechtecke**

DIRECT Step 2: Bestimme potentiell optimale Rechtecke

- Ein centerpoint $\vec{c}_j$ ist pot. opt., wenn ein $\tilde{k} > 0$ existiert mit
 - $f(\vec{c}_j) - \tilde{k}d_j \leq f(\vec{c}_i) - \tilde{k}d_i$
für alle $i \in 1, \dots, m$
 - $f(\vec{c}_j) - \tilde{k}d_j \leq f_{\min} - \epsilon|f_{\min}|$
- d_i : Distanz von centerpoint zu Ecke des Rechtecks
- ϵ : kleine positive Zahl

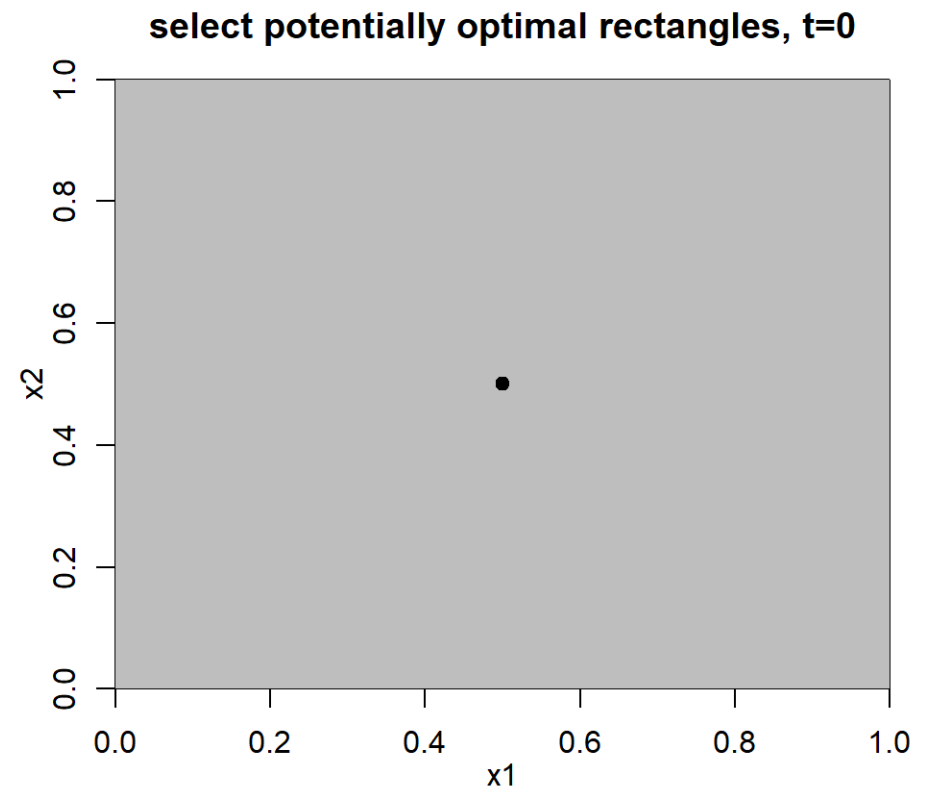
DIRECT Step 2: Bestimme potentiell optimale Rechtecke

- Motivation: sowohl lokal als auch global optimieren (Balance: exploitation vs. exploration)
- Potentiell optimale Punkte sind auf konvexer Hülle, s. Abbildung
- d_i bedeutet Größe der Rechtecke, $f(\vec{c}_i)$ ihre Güte



DIRECT Step 2: Bestimme potentiell optimale Rechtecke

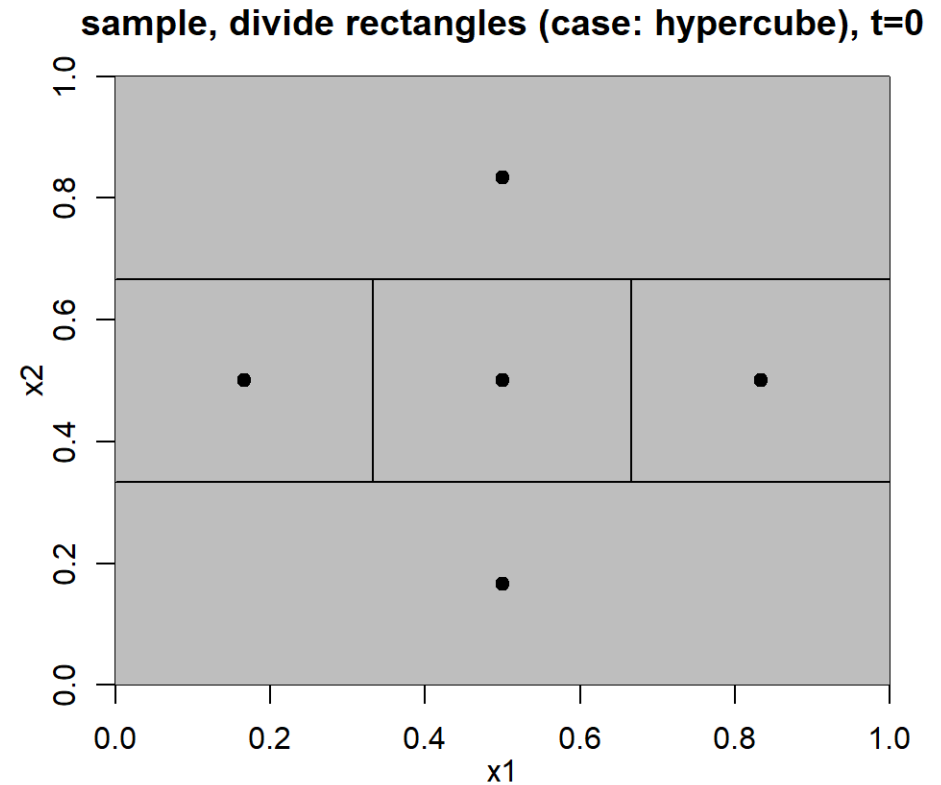
- Trivial in erster Iteration, $t = 0$: nur ein Rechteck auszuwählen



DIRECT: Sample & Divide

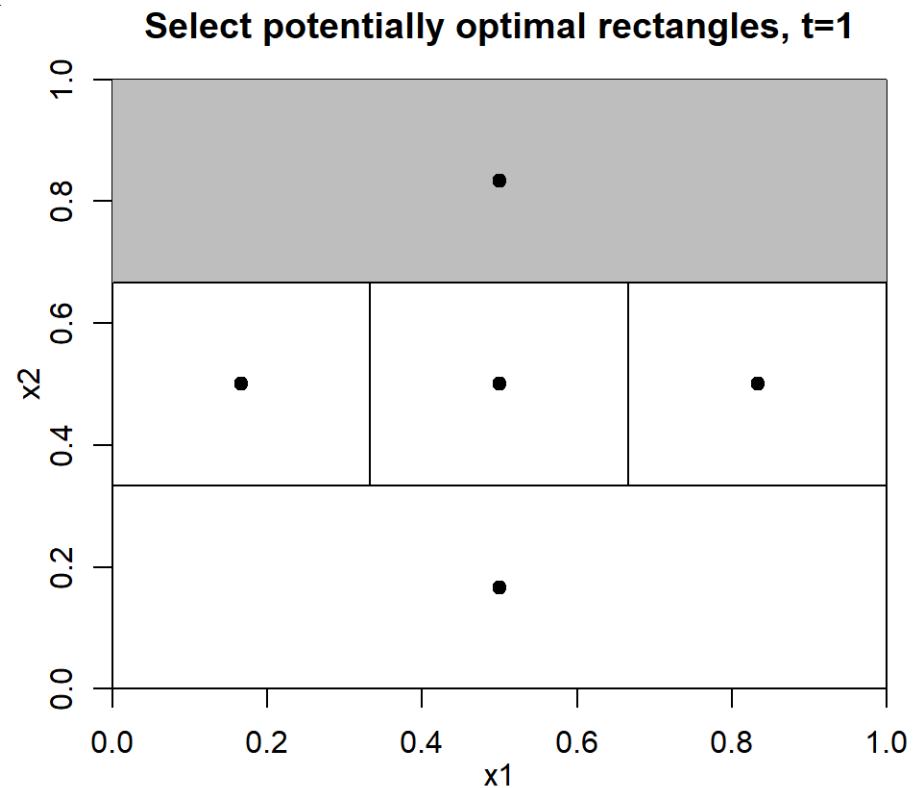
DIRECT Step 3: Sample & Divide

- Bestimme Menge I von Dimensionen mit maximaler Seitenlänge 3δ .
- **Sample** (neue Punkte setzen) bei $\vec{c} \pm \delta\vec{e}_i$, für alle $i \in I$
- **Divide** bzw. aufteilen des Rechtecks in Drittel, entlang aller Dimensionen in I , beginnend mit $i \in I$ welches den besten Wert $f(\vec{c}_i)$ liefert
- Beispiel in Abb.: zwei Dimensionen mit gleicher Seitenlänge (Quadrat)



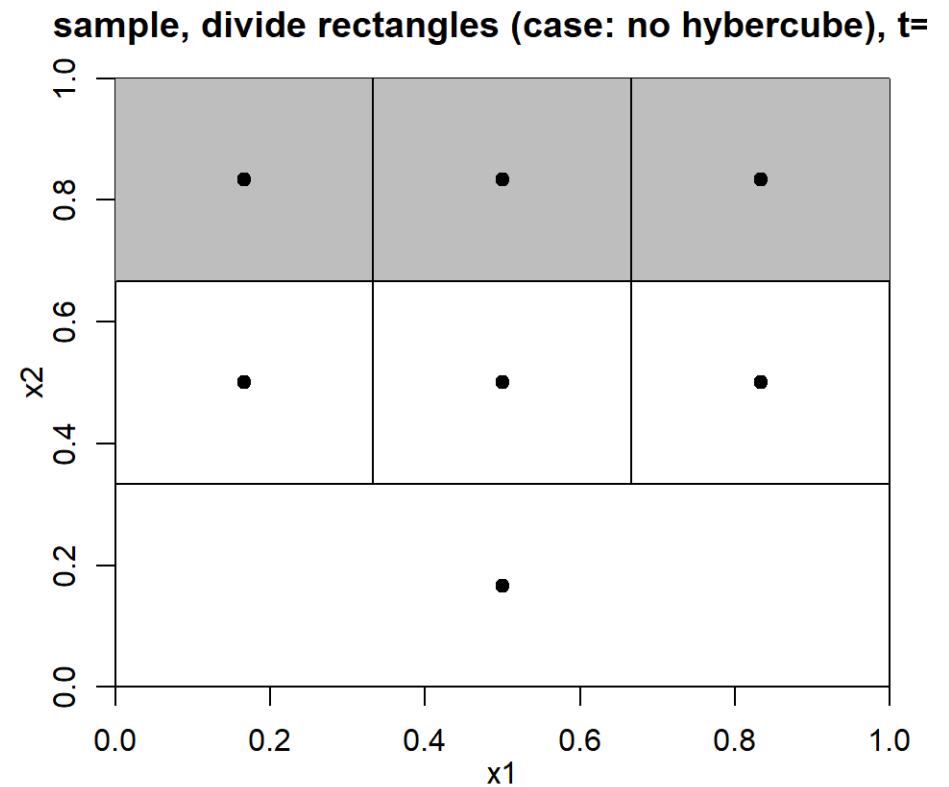
DIRECT Step 2: Bestimme potentiell optimale Rechtecke

- Beispiel in Abb.: einzelnes pot. opt. Rechteck



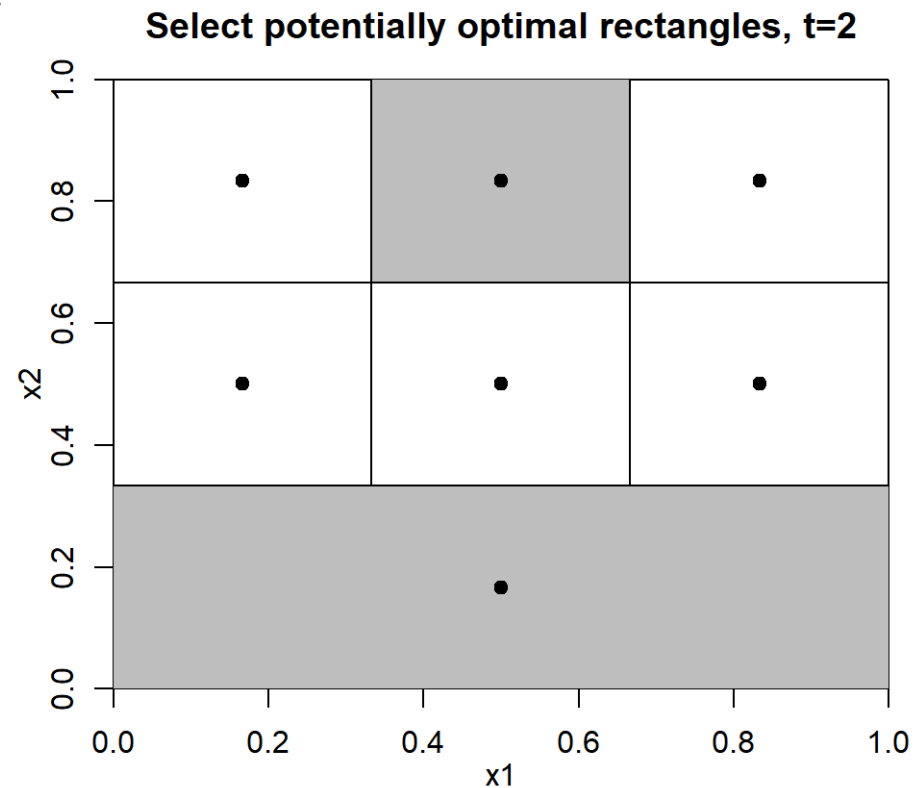
DIRECT Step 3: Sample & Divide

- Beispiel in Abb.: teile einzelne größte Dimension des gewählten Rechtecks



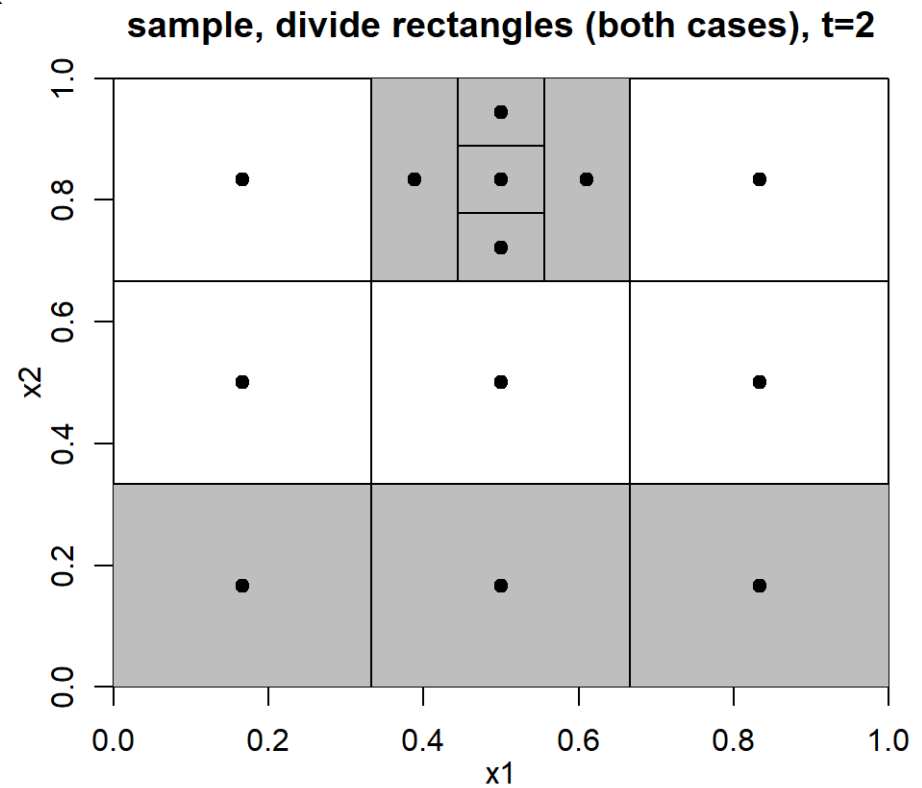
DIRECT Step 2: Bestimme potentiell optimale Rechtecke

- Beispiel in Abb.: zwei Rechtecke sind pot. opt.



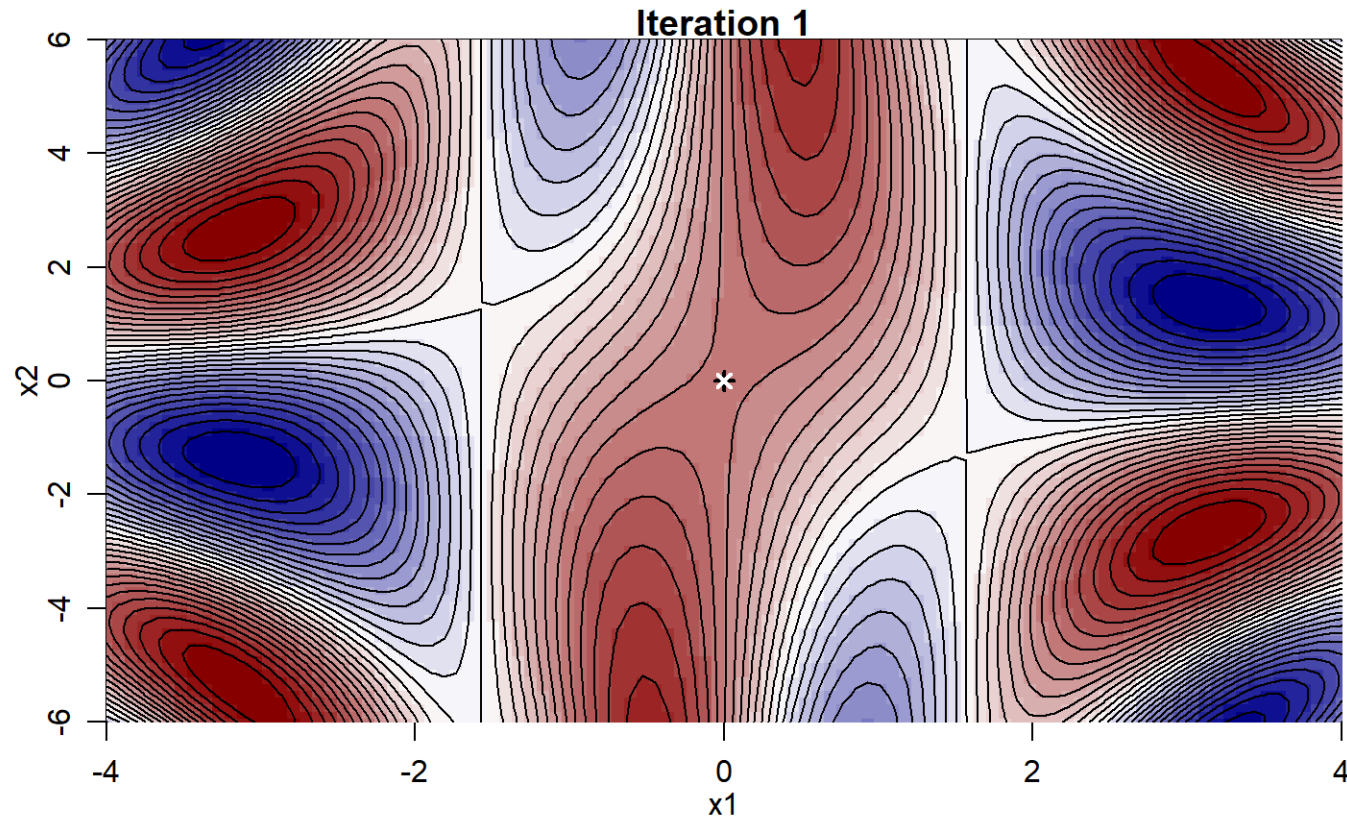
DIRECT Step 3: Sample & Divide

- Beispiel in Abb.: teile jedes pot. opt. Rechteck

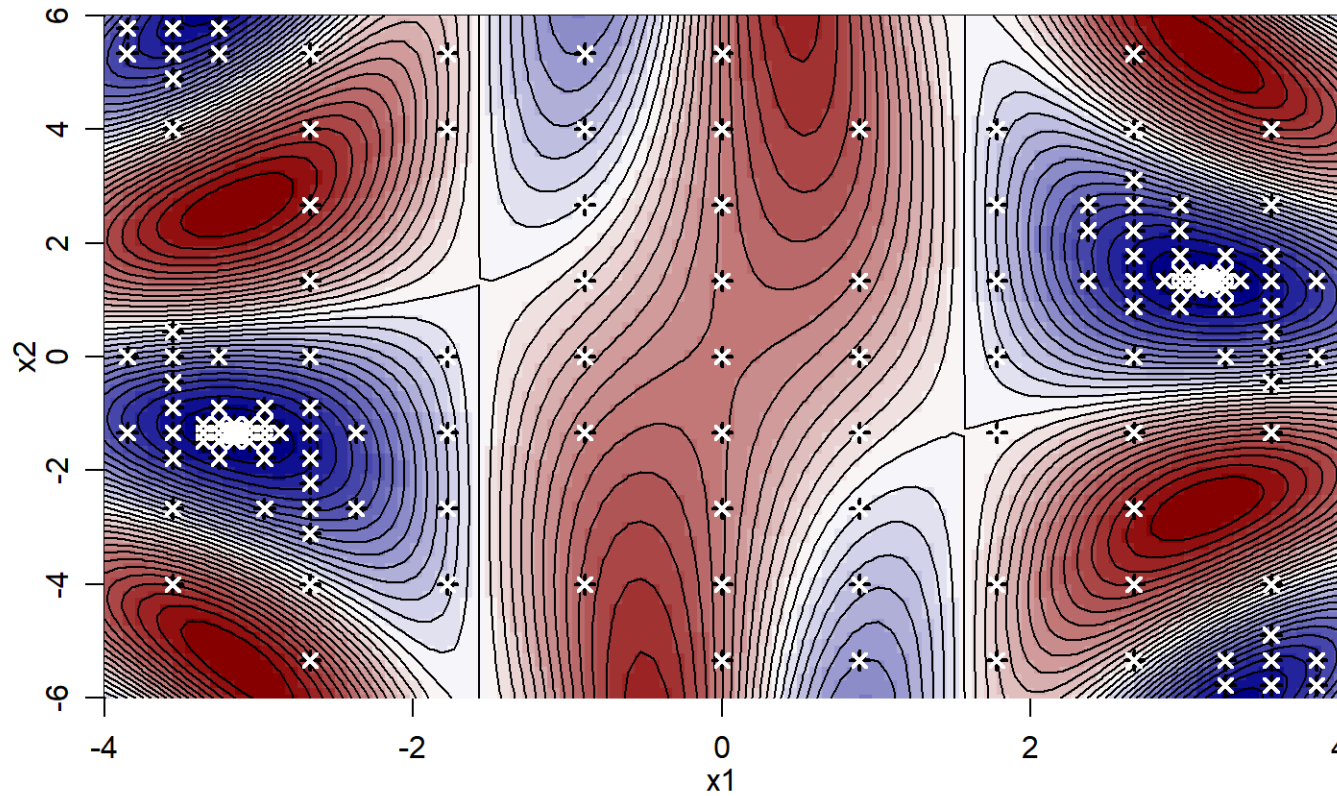


DIRECT: Beispiel und Überblick

DIRECT: Beispiel



Nach 200 iterationen



Überblick

Eigenschaften von Algorithmen/Methoden

Methode	Global	Gradientenbasiert	Nebenbedingungen	Ziele	Linear	Ansatz	Genauigkeit	Deterministisch	Variablen
Stationäre Punkte / Hesse	✓	✓	ohne	1	✗	analytisch	exakt	✓	$\mathbb{R}^n$
Gradient Descent	✗	✓	ohne	1	✗	numerisch	?	✓	$\mathbb{R}^n$
Simplex	✗	✓	alle	1	✓	numerisch	exakt	✓	$\mathbb{R}^n$ & $\mathbb{Z}^n$
QO	✗	✓	alle	1	Nebenbed.	numerisch	?	✓	$\mathbb{R}^n$
KKT	✓	✓	alle	1		analytisch	exakt	✓	$\mathbb{R}^n$
SQO	✗	✓	alle	1		numerisch	?	✓	$\mathbb{R}^n$
Nelder-Mead	✗	✗	ohne	1	✗	numerisch	approx.	✓	$\mathbb{R}^n$
DIRECT	✓	✗	Grenzen	1	✗	numerisch	approx.	✓	$\mathbb{R}^n$

R Anwendung

- Beispiel zeigt wie DIRECT in R verwendet wird
- Aufgabe: Vergleich mit Nelder Mead Algorithmus + **restarts** aus voriger Anwendung
- Siehe Details in AnwendungOV-05b-DIRECT.Rmd